PYTHON KURSU • HAFTA 8 • GÜN 2

OOP 1: Sınıflar ve Nesneler

Ders 16 / 32 • 3 saat • VS Code + Python 3.12

Bu dersin sonunda…

- Sınıf ve nesne kavramlarını günlük hayattan örneklerle açıklayabileceksiniz.
- class, __init__ ve self ile kendi sınıfınızı tanımlayabileceksiniz.
- Nesnenin durumunu değiştiren ve değer döndüren metotlar yazabileceksiniz.
- __str__ ile nesneleri okunaklı yazdırabileceksiniz.
- Banka hesabı ve alışveriş sepeti gibi küçük nesne modelleri kurabileceksiniz.

AÇILIŞ • 10 DK

Bugünün zaman planı

Saat	Süre	Bölüm
0:00–0:10	10 dk	Açılış: hedefler, Ders 15'in tekrarı
0:10–1:25	75 dk	Blok 1: zaten nesne kullanıyoruz • neden OOP • sınıf ve nesne • __init__ ve self • varsayılan değerler • metotlar
1:25–1:40	15 dk	Mola
1:40–2:55	75 dk	Blok 2: banka hesabı • hareket geçmişi • __str__ • sınıf değişkenleri • nesne listeleri • alışveriş sepeti • alıştırmalar
2:55–3:00	5 dk	Ödev ve gelecek ders

AÇILIŞ • DERS 15'IN TEKRARI • 6 DK

Dört soruda geçen ders

SORU 1 • Ne gelir?

```
SELECT ad FROM kitaplar
WHERE yil = NULL
```

SORU 2 • Ne olur?

```
UPDATE kitaplar SET puan = 1
-- WHERE yok
```

SORU 3 • Neyi bulur?

```
WHERE ad LIKE '%ince%'
-- kayıt: 'İnce Memed'
```

SORU 4 • Ne işe yarar?

```
INSERT OR IGNORE INTO …
-- UNIQUE (ad, yazar)
```

AÇILIŞ • ÖDEV KONTROLÜ • 4 DK

Ödev 1: film arşivinden arama fonksiyonu

```
def film_ara(baglanti, metin):
    kalip = f"%{tr_kucuk(metin)}%"
    return baglanti.execute(
        """SELECT id, ad, yonetmen, yil FROM filmler
           WHERE tr_kucuk(ad) LIKE ?
              OR tr_kucuk(yonetmen) LIKE ?
           ORDER BY yil""", (kalip, kalip)).fetchall()
```

DİKKAT

Her fonksiyonda baglanti

film_ara, film_ekle, puan_guncelle, film_sil… hepsi aynı parametreyi taşıyor.

BUGÜN

Veri + davranış bir arada

Bağlantı bir nesnenin içinde dursa, fonksiyonlar onun metotları olsa?

BLOK 1 • 0:10–1:25 • 75 DK

Sınıflar ve nesneler

15 dk **Neden OOP?**	25 dk **class ve __init__**	25 dk **Metotlar**	10 dk **Hatalar**

BLOK 1 • NEDEN OOP? • 15 DK

Aslında baştan beri nesne kullanıyoruz

```
metin = "merhaba"
liste = [3, 1, 2]
print(type(metin), type(liste))
print(metin.upper(), metin.count("a"))
liste.append(5)
liste.sort()
print(liste)
```

```
ÇIKTI
<class 'str'> <class 'list'>
MERHABA 2
[1, 2, 3, 5]
```

metin, str sınıfının bir nesnesi; **upper()** onun metodu. **liste.append(5)** listenin kendisini değiştiriyor.

BLOK 1 • NEDEN OOP? • 15 DK

Sözlük + fonksiyonlarla model kurmanın sınırı

```python
urun = {"ad": "Defter", "fiyat": 45.0, "stok": 10}

def stok_dus(urun, adet):
    urun["stok"] -= adet

def indirimli(urun, oran):
    return urun["fiyat"] * (1 - oran)

stok_dus(urun, 3)
print(urun["stok"], indirimli(urun, 0.2))
```

SORUN 1

Dağınıklık

Veri bir yerde, onu işleyen fonksiyonlar başka yerde.

SORUN 2

Kural yok

urun["stok"] = -50 yazılabilir; yazım hatası ("stk") fark edilmez.

BLOK 1 • NEDEN OOP? • 15 DK

Sınıf bir kalıp, nesne o kalıptan çıkan örnek

Benzetme: kurabiye kalıbı

Kalıp (sınıf) bir kez yapılır; ondan istediğiniz kadar kurabiye (nesne) çıkar. Her kurabiyenin şekli aynıdır ama birinin üstü çikolatalı, diğeri susamlı olabilir (farklı öznitelik değerleri).

Kavram	Ürün örneğinde
Sınıf (class)	Urun: her ürünün ne taşıdığı
Nesne / örnek	defter, kalem, çanta
Öznitelik (attribute)	ad, fiyat, stok
Metot (method)	stok_dus(), indirimli()

BLOK 1 • CLASS VE __INIT__ • 25 DK

İlk sınıfımız: class Urun

```
class Urun:
    pass

defter = Urun()
defter.ad = "Defter"
defter.fiyat = 45.0
print(type(defter))
print(defter.ad, defter.fiyat)
```

```
ÇIKTI
<class '__main__.Urun'>
Defter 45.0
```

Çalışıyor ama her nesneye öznitelikleri tek tek eklemek zahmetli ve unutulmaya açık. Çözüm: **__init__**.

BLOK 1 • CLASS VE __INIT__ • 25 DK

__init__: nesne doğarken çalışan metot

```
class Urun:
    def __init__(self, ad, fiyat, stok):
        self.ad = ad
        self.fiyat = fiyat
        self.stok = stok

defter = Urun("Defter", 45.0, 10)
kalem = Urun("Kalem", 12.5, 40)
print(defter.ad, defter.fiyat, defter.stok)
print(kalem.ad, kalem.fiyat, kalem.stok)
```

```
ÇIKTI
Defter 45.0 10
Kalem 12.5 40
```

Urun("Defter", 45.0, 10) → Python yeni bir nesne oluşturur ve **__init__(nesne, "Defter", 45.0, 10)** çağırır.

BLOK 1 • CLASS VE __INIT__ • 25 DK

self: “bu nesne” demek

Siz yazarsınız	Python’un yaptığı	self kimdir?
`defter = Urun("Defter", 45.0, 10)`	`Urun.__init__(defter, …)`	`defter`
`kalem = Urun("Kalem", 12.5, 40)`	`Urun.__init__(kalem, …)`	`kalem`
`defter.stok_dus(3)`	`Urun.stok_dus(defter, 3)`	`defter`

Benzetme

Bir formdaki “Adınız:” satırı. Formu kim dolduruyorsa “ad” onun adıdır. self, metodu çağıran nesnenin yerini tutar.

BLOK 1 • CLASS VE __INIT__ • 25 DK

Varsayılan değerler ve isimli argümanlar

```
class Urun:
    def __init__(self, ad, fiyat, stok=0, kategori="Genel"):
        self.ad = ad
        self.fiyat = fiyat
        self.stok = stok
        self.kategori = kategori

silgi = Urun("Silgi", 7.5)
canta = Urun("Çanta", 650.0, stok=4, kategori="Aksesuar")
print(silgi.ad, silgi.stok, silgi.kategori)
print(canta.ad, canta.stok, canta.kategori)
```

```
ÇIKTI
Silgi 0 Genel
Çanta 4 Aksesuar
```

Ders 9’daki fonksiyon parametreleri kuralları aynen geçerli: varsayılanlı parametreler sonda.

BLOK 1 • METOTLAR • 25 DK

Metotlar: nesnenin yapabildikleri

```
class Urun:
    def __init__(self, ad, fiyat, stok=0):
        self.ad = ad
        self.fiyat = fiyat
        self.stok = stok

    def indirimli_fiyat(self, oran):
        return round(self.fiyat * (1 - oran), 2)

    def stok_dus(self, adet):
        if adet > self.stok:
            raise ValueError(f"Yetersiz stok: {self.stok} adet var.")
        self.stok -= adet

    def stok_degeri(self):
        return self.fiyat * self.stok
```

```
defter = Urun("Defter", 45.0, 10)
print(defter.indirimli_fiyat(0.2))
defter.stok_dus(3)
print(defter.stok, defter.stok_degeri())
```

```
ÇIKTI
36.0
7 315.0
```

BLOK 1 • METOTLAR • 25 DK

defter.stok_dus(2) aslında ne demek?

```
defter = Urun("Defter", 45.0, 10)
defter.stok_dus(2)
Urun.stok_dus(defter, 2)  # aynı işlem, uzun yazım
print(defter.stok)
```

```
ÇIKTI
6
```

İki çağrı da aynı nesnenin stoğunu düşürdü: 10 → 8 → 6.

Kural

nesne.metot(argümanlar) ≡ Sınıf.metot(nesne, argümanlar). Kısa yazımı kullanırız; uzun yazım self'in nereden geldiğini gösterir.

BLOK 1 • SIK YAPILAN HATALAR

Sınıf yazarken en sık hatalar

Hatalı kod	Hata mesajı	Çözüm
`def etiket(): # self yok`	`TypeError: Urun.etiket() takes 0 positional arguments but 1 was given`	ilk parametre self
`stok -= adet # self. yok`	`UnboundLocalError: cannot access local variable 'stok' where it is not associated with a value`	self.stok -= adet
`Urun("Defter") # fiyat eksik`	`TypeError: Urun.__init__() missing 1 required positional argument: 'fiyat'`	bütün zorunlu argümanlar
`self.fiyat atanmadı`	`AttributeError: 'Urun' object has no attribute 'fiyat'`	__init__'te self.fiyat = fiyat

1:25–1:40 • 15 DK

Mola

Moladan sonra: bir banka hesabını sınıf olarak nasıl modelleriz?

BLOK 2 • 1:40–2:55 • 75 DK

Gerçekçi nesne modelleri

25 dk
Banka hesabı

10 dk
Sınıf değişkenleri

22 dk
Nesne listeleri ve sepet

18 dk
Alıştırmalar

BLOK 2 • BANKA HESABI • 25 DK

BankaHesabi: kurallar sınıfın içinde

```python
class BankaHesabi:
    def __init__(self, sahip, bakiye=0.0):
        self.sahip = sahip
        self.bakiye = bakiye

    def yatir(self, tutar):
        if tutar ⩽ 0:
            raise ValueError("Tutar pozitif olmalı.")
        self.bakiye += tutar

    def cek(self, tutar):
        if tutar ⩽ 0:
            raise ValueError("Tutar pozitif olmalı.")
        if tutar > self.bakiye:
            raise ValueError(f"Yetersiz bakiye ({self.bakiye:.2f} TL).")
        self.bakiye -= tutar
```

ÖZNİTELİK • METOT

sahip, bakiye • yatir, cek

Bakiyeyi yalnızca bu metotlar değiştirir.

KURAL

ValueError

Negatif tutar ve eksi bakiye reddedilir.

BLOK 2 • BANKA HESABI • 25 DK

İki hesap, iki ayrı bakiye

```
ayse = BankaHesabi("Ayşe Yılmaz", 1000)
mehmet = BankaHesabi("Mehmet Kaya")
ayse.yatir(250)
mehmet.yatir(400)
ayse.cek(300)
print(ayse.sahip, ayse.bakiye)
print(mehmet.sahip, mehmet.bakiye)
for islem, tutar in [(mehmet.cek, 1000), (ayse.yatir, -50)]:
    try:
        islem(tutar)
    except ValueError as hata:
        print("Reddedildi:", hata)
```

```
ÇIKTI
Ayşe Yılmaz 950
Mehmet Kaya 400.0
Reddedildi: Yetersiz bakiye (400.00 TL).
Reddedildi: Tutar pozitif olmalı.
```

BLOK 2 • BANKA HESABI • 25 DK

Nesnenin içinde liste: hareket geçmişi

```
    def __init__(self, sahip, bakiye=0.0):
        self.sahip = sahip
        self.bakiye = bakiye
        self.hareketler = []    # her nesnenin kendi listesi

    def yatir(self, tutar):
        ...
        self.bakiye += tutar
        self.hareketler.append((tarih, "Yatırma", tutar))

    def ekstre(self):
        for tarih, tur, tutar in self.hareketler:
            print(f"  {tarih}  {tur:<8}{tutar:>9.2f}")
```

```
ÇIKTI: hesap.ekstre()
Hesap: Ayşe Yılmaz
  2026-10-26  Yatırma    250.00
  2026-10-26  Çekme     -300.00
  Bakiye: 950.00 TL
```

BLOK 2 • BANKA HESABI • 25 DK

__str__: nesne yazdırılınca ne görünsün?

```
class Urun:
    ...
    def __str__(self):
        return f"{self.ad} — {self.fiyat:.2f} TL ({self.stok} adet)"

print(UrunEski("Defter"))      # __str__ yok
defter = Urun("Defter", 45, 10)
print(defter)
print(f"Sepete eklendi: {defter}")
```

Özel metotlar

Adı __ile__ başlayıp biten metotları Python belirli durumlarda kendisi çağırır: __init__ oluştururken, __str__ yazdırırken.

```
ÇIKTI
<__main__.UrunEski object at 0x7f…>
Defter — 45.00 TL (10 adet)
Sepete eklendi: Defter — 45.00 TL (10 adet)
```

BLOK 2 • SINIF DEĞIŞKENLERI • 10 DK

Sınıf değişkeni ve örnek değişkeni

```
class BankaHesabi:
    banka = "Python Bankası"     # sınıf değişkeni: hepsi paylaşır
    hesap_sayisi = 0

    def __init__(self, sahip):
        self.sahip = sahip           # örnek değişkeni: her nesneye ait
        BankaHesabi.hesap_sayisi += 1
        self.no = BankaHesabi.hesap_sayisi


a = BankaHesabi("Ayşe")
b = BankaHesabi("Mehmet")
print(a.banka, "|", a.sahip, a.no, "|", b.sahip, b.no)
print("Toplam hesap:", BankaHesabi.hesap_sayisi)
```

Tür	Kimin?
Sınıf değişkeni (class altında)	herkesin ortak
Örnek değişkeni (self.x = …)	her nesnenin kendi

```
ÇIKTI
Python Bankası | Ayşe 1 | Mehmet 2
Toplam hesap: 2
```

BLOK 2 • NESNE LISTELERI VE SEPET • 22 DK

Nesnelerden oluşan listeler

```
urunler = [Urun("Defter", 45.0, 10), Urun("Kalem", 12.5, 40),
           Urun("Çanta", 650.0, 2), Urun("Silgi", 7.5, 0)]
toplam = sum(u.stok_degeri() for u in urunler)
print(f"Toplam stok değeri: {toplam:.2f} TL")
print("Stokta olmayanlar:", [u.ad for u in urunler if u.stok == 0])
en_pahali = urunler[0]
for u in urunler:
    if u.fiyat > en_pahali.fiyat:
        en_pahali = u
print("En pahalı:", en_pahali.ad)
```

```
ÇIKTI
Toplam stok değeri: 2250.00 TL
Stokta olmayanlar: ['Silgi']
En pahalı: Çanta
```

BLOK 2 • NESNE LISTELERI VE SEPET • 22 DK

Uygulama: alışveriş sepeti

```
Sepet (fis metodu .py dosyasında)
class Sepet:
    def __init__(self, musteri):
        self.musteri = musteri
        self.kalemler = []        # (Urun, adet) demetleri

    def ekle(self, urun, adet=1):
        self.kalemler.append((urun, adet))

    def toplam(self):
        return sum(urun.fiyat * adet
                   for urun, adet in self.kalemler)

    def indirim_uygula(self, kod):
        indirimler = {"OGRENCI10": 0.10, "YILBASI20": 0.20}
        oran = indirimler.get(kod.upper(), 0)
        return round(self.toplam() * (1 - oran), 2), oran
```

NESNE İÇİNDE NESNE

Sepet → Urun

Sepetin kalemleri Urun nesneleri taşır; toplamı ürünlerin fiyatından hesaplar.

METOT METOT ÇAĞIRIR

self.toplam()

indirim_uygula, aynı nesnenin toplam metodunu kullanır.

BLOK 2 • NESNE LISTELERI VE SEPET • 22 DK

Sepeti kullanmak

```
defter = Urun("Defter", 45.0)
kalem = Urun("Kalem", 12.5)
sepet = Sepet("Zeynep")
sepet.ekle(defter, 3)
sepet.ekle(kalem, 4)
sepet.fis("ogrenci10")
```

```
ÇIKTI
Müşteri: Zeynep
  3 x Defter     135.00
  4 x Kalem       50.00
  Ara toplam: 185.00  İndirim: %10
  Ödenecek: 166.50 TL
```

BLOK 2 • SIK YAPILAN HATALAR

Hata vermeyen ama yanlış sonuç veren durumlar

Kod	Ne görünür?	Doğrusu
`print(h.bakiye_goster)`		`h.bakiye_goster()` — parantez
`print(defter) # __str__ yok`	`<__main__.Urun object at 0x7f…>`	`__str__` metodu yazın
`self.hesap_sayisi += 1`	ortak sayaç hep 0 kalır	`BankaHesabi.hesap_sayisi += 1`
`class altında: hareketler = []`	bütün hesaplar aynı listeyi paylaşır	`__init__'te self.hareketler = []`
`def __str__(self): print(…)`	`TypeError: __str__ returned non-string`	`return f"…"`

BLOK 2 • SINIF TASARLAMAK

Bir sınıfı nasıl tasarlarız?

Problemi bir cümleyle anlatın

“Bir **öğrenci**nin **adı**, **numarası** ve **notları** vardır. Öğrenciye **not eklenir**, **ortalaması hesaplanır** ve **geçip geçmediği** söylenir.”

Cümlede	**Kodda**
isimler (öğrenci)	sınıf: Ogrenci
özellik isimleri	öznitelik: ad, numara, notlar
fiiller	metot: not_ekle, ortalama, durum

BLOK 2 • SINIF IÇI ALIŞTIRMALAR • 18 DK

Alıştırma 1, 2 ve 3

KOLAY • 4 dk

1 • Dikdörtgen

Dikdortgen sınıfı: en ve boy öznitelikleri; alan() ve cevre() metotları; kare_mi() True/False döndürsün.

ORTA • 6 dk

2 • Öğrenci

Ogrenci(ad, numara): not_ekle(not_) (0–100 dışı ValueError), ortalama(), durum() → “Geçti” (≥ 50) / “Kaldı”; __str__.

ZOR • 8 dk

3 • Akıllı sepet

Sepet’e: aynı ürün tekrar eklenince adedi artsın; cikar(urun_adi) ve urun_sayisi() metotları ekleyin.

BLOK 2 • ALIŞTIRMA ÇÖZÜMLERI

Çözüm 1: dikdörtgen

```
class Dikdortgen:
    def __init__(self, en, boy):
        self.en = en
        self.boy = boy

    def alan(self):
        return self.en * self.boy

    def cevre(self):
        return 2 * (self.en + self.boy)

    def kare_mi(self):
        return self.en == self.boy


masa = Dikdortgen(120, 80)
fayans = Dikdortgen(30, 30)
print(masa.alan(), masa.cevre(), masa.kare_mi())
print(fayans.alan(), fayans.cevre(), fayans.kare_mi())
```

```
ÇIKTI
9600 400 False
900 120 True
```

BLOK 2 • ALIŞTIRMA ÇÖZÜMLERI

Çözüm 2: öğrenci

```
class Ogrenci:
    def __init__(self, ad, numara):
        self.ad = ad
        self.numara = numara
        self.notlar = []
    def not_ekle(self, not_):
        if not 0 ⩽ not_ ⩽ 100:
            raise ValueError("Not 0 ile 100 arasında olmalı.")
        self.notlar.append(not_)
    def ortalama(self):
        if not self.notlar:
            return 0
        return sum(self.notlar) / len(self.notlar)
    def durum(self):
        return "Geçti" if self.ortalama() ⩾ 50 else "Kaldı"
    def __str__(self):
        ort = self.ortalama()
        return f"{self.numara} {self.ad}: {ort:.1f} ({self.durum()})"
```

```
ece = Ogrenci("Ece", 1024)
for n in [72, 45, 88]:
    ece.not_ekle(n)
print(ece)
ece.not_ekle(120)  # try içinde
```

```
ÇIKTI
1024 Ece: 68.3 (Geçti)
Hata: Not 0 ile 100 arasında
  olmalı.
```

BLOK 2 • ALIŞTIRMA ÇÖZÜMLERI

Çözüm 3: akıllı sepet

```
class Sepet:
    def __init__(self):
        self.kalemler = {}            # ürün adı → [Urun, adet]

    def ekle(self, urun, adet=1):
        if urun.ad in self.kalemler:
            self.kalemler[urun.ad][1] += adet
        else:
            self.kalemler[urun.ad] = [urun, adet]

    def cikar(self, urun_adi):
        if self.kalemler.pop(urun_adi, None) is None:
            print(f"{urun_adi} sepette yok.")

    def urun_sayisi(self):
        return sum(adet for _, adet in self.kalemler.values())

    def toplam(self):
        return sum(urun.fiyat * adet
                   for urun, adet in self.kalemler.values())
```

```
sepet.ekle(defter, 2)
sepet.ekle(kalem)
sepet.ekle(defter)
print(sepet.urun_sayisi(),
      sepet.toplam())
sepet.cikar("Kalem")
sepet.cikar("Silgi")
```

```
ÇIKTI
4 147.5
Silgi sepette yok.
3 135.0
```

KAPANIŞ • TERIMLER

Bugünün sözlüğü

Terim	Anlamı	Örnek
sınıf (class)	kalıp / plan	`class Urun:`
nesne, örnek (instance)	sınıftan üretilen somut şey	`defter = Urun(…)`
öznitelik (attribute)	nesnenin verisi	`defter.fiyat`
metot (method)	sınıftaki fonksiyon	`defter.stok_dus(3)`
__init__	nesne oluşurken çalışır	`def __init__(self, …)`
self	metodu çağıran nesne	`self.stok -= adet`
__str__	yazdırılınca görünen metin	`print(defter)`
sınıf değişkeni	bütün nesnelerin ortak verisi	`BankaHesabi.banka`

KAPANIŞ • 5 DK

Bugün ne öğrendik?

- **Sınıf** kalıptır, **nesne** ondan üretilir; veri (öznitelik) ve davranış (metot) bir arada.
- **__init__(self, ...)** nesne oluşurken çalışır; öznitelikler **self.x = x** ile kaydedilir.
- Metotların ilk parametresi **self**; nesne.metot(a) ≡ Sınıf.metot(nesne, a).
- Kurallar metotların içinde: hatalı işlemde **raise ValueError**.
- **__str__** okunaklı yazdırma; ortak veriler **sınıf değişkeni**, kişisel listeler **__init__**'te.

KAPANIŞ • ÖDEV • TESLIM: BIR SONRAKI DERS

Ödev

1 • ZORUNLU

Kitap ve Kitaplık

Kitap(ad, yazar, yil, puan) ve Kitaplik sınıfları: ekle, ara(metin), puanla(ad, puan), en_iyiler(n). Kitap'a __str__ yazın.

2 • ZORUNLU

Kendi nesneniz

Molada tasarladığınız nesneyi (telefon, oyun karakteri…) sınıf olarak yazın: en az 3 öznitelik, 3 metot, 1 kural (ValueError).

3 • BONUS

SQLite'lı Kitaplık

Ders 15 mini projesini sınıfa çevirin: KitaplikDB; bağlantı __init__'te açılsın, ara/guncelle/sil metot olsun.

Teslim: ders16_odev.py ya da .ipynb. Teslim yöntemi: [teslim yöntemi]

KAPANIŞ • GELECEK DERS: HAFTA 9 • GÜN 1

OOP 2: miras ve kapsülleme

Bir yönetici de bir çalışandır: maaşı, adı vardır, ama ek olarak ekibi ve primi vardır. Gelecek ders:

- Miras: class Yonetici(Calisan)
- super() ile üst sınıfı kullanmak
- Metotları yeniden tanımlamak (override)
- Kapsülleme: _ ve __, @property • mini proje

Önizleme

```
class Calisan:
    def __init__(self, ad, maas):
        self.ad, self.maas = ad, maas

class Yonetici(Calisan):
    pass

y = Yonetici("Ayşe", 85000)
print(y.ad, isinstance(y, Calisan))
```

```
Ayşe True
```

KAYNAKLAR

Bu dersin konuları resmi belgelerde

Ne için?	Adres
Python belgeleri: Sınıflar	docs.python.org/3/tutorial/classes.html
Özel metotlar (data model)	docs.python.org/3/reference/datamodel.html
PEP 8: adlandırma kuralları	peps.python.org/pep-0008
Real Python: OOP	realpython.com/python3-object-oriented-programming